1. Has-a relationships can be seen when a class contains or uses the instance of another class, while is-a relationships are seen when classes(subclass/child class) are derived from existing classes(superclass/parent class); subclasses inherit behaviours and properites from the superclass. Has-a relationships can be described like a library has books, while is-a relationships are described as how an apple is a fruit.

2. Both go() and stop() methods will be available to an object created by the subclass; go() is public, meaning that the subclass can inherit the method.

3. When a subclass inherits from an abstract class with abstract methods, the subclass must implement the inherited abstract method(is mandatory). Overriding a method is when a subclass provides a new implementation for an inherited method and is not mandatory. Abstract methods do not have body while the parent method that is to be overridden may have a body.

4. Objects cannot be created from both abstract class and interfaces, however, interfaces cannot be inherited unlike abstract classes. Abstract classes uses is-a relationships to define what the object is, while interfaces uses has-a relationships to define what an object can do, without defining how. For example, both airplanes and birds can fly, but how they fly are different. Abstract classes are used to define a common identity, like how both cats and dogs are animals.

6a. *doThat()* is an abstract method as it doesn't have a body.

b. *Wo* is an interface as it uses the keyword *interface* when declaring the class.

c. *Roo* implements the *Wo* class, and because *doThat()* is an abstract method, it must be implemented in *Roo*.

d. *doThis()* and *doNow(). doThat()* cannot be used by the object as it has a private modifier.